Test

Loss functions for neural networks

Neural networks are non-linear parametric prediction models. All weights and biases that transform one layer into the next can be collected into a single long parameter vector θ, even if this vector has thousands or hundreds of thousands of entries. For a fixed θ the network defines a predictive function, which we denote by f. In regression we usually write $f(x)$ and keep in mind that it depends on θ, while in classification with q terminal neurons the network outputs q functions $f_1(x), \dots, f_q(x)$, one for each class probability.

Given training data $(x_1, y_1), \dots, (x_n, y_n)$, we define a loss function $J(\theta)$ that measures how well the network predicts the responses on the whole training set. This $J(\theta)$ is our objective function: if we change θ, the neural network changes, the predictions $f(x_i)$ change, and therefore the value of $J(\theta)$ changes.

For regression, where the y_i are quantitative, we use the usual squared-error loss and define the residual sum of squares

$$J(\theta) = \sum_{i=1}^{n} \big(y_i - f(x_i)\big)^2.$$

This is just the squared error iterated over all training samples, comparing each true response to the prediction from the neural network.

For classification with q classes, $y_i \in \{1, \dots, q\}$, we use the cross-entropy (deviance) loss known from multinomial logistic regression,

$$J(\theta) = -\sum_{i=1}^{n} \sum_{j=1}^{q} I\{y_i = j\} \log f_j(x_i).$$

Here $I\{y_i = j\}$ is the indicator of the true class. For each observation this loss takes the log-probability that the network assigns to the true class and sums it over all training samples. Minimising $J(\theta)$ in this case is equivalent to maximum likelihood estimation of a classifier with class probabilities given by the neural network.